

Step 3 — Scaling to 3+ Tools With a Hard Step Limit and Failure Recovery — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper LangChain), **Advanced** (what actually makes this the "real Worker" the README promises, not just Step 2 with more tools bolted on). Read Basic first even if you already know the pattern — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

Step 2's loop already has the right shape — ask, check for tool calls, run or stop. This step doesn't change that shape at all. It adds three things around it: more tools to choose from, a hard ceiling on how many rounds the loop can run, and a way for a tool that actually fails to not take the whole agent down with it.

The step limit is the part to get right first, conceptually: it has to live in *your* Python code (a counter you check yourself), never in a prompt asking the model nicely to stop. A model can't reliably enforce a limit on its own behavior — only code outside the loop can guarantee it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Grow Step 2's `run_agent()` into:

```
def run_agent(task: str, max_iterations: int = 6) -> AgentResult:
```

Three concrete additions: - **More tools.** At least 2 more beyond Step 2's calculator, at least one backed by a real service (reuse Doc02's client — a weather lookup, a search call, anything that actually hits a network), bound together: `model.bind_tools([calculator, lookup_weather, flaky_tool])`. - **A round counter.** Increment it once per loop iteration (not per tool call — a single round can include several tool calls if the model asks for more than one at once). When it exceeds `max_iterations`, raise `MaxIterationsExceeded`, a custom exception, carrying the partial `steps` log collected so far — never just quietly return an incomplete answer. - **A tool built to fail sometimes.** Not randomly-flaky in a way you can't reproduce — deterministically, on a specific input you control (like raising on a particular argument value), so your test for "does the agent recover" is actually repeatable.

The recovery behavior itself is the same `try/except` pattern Step 2's Advanced hint already introduced — this step's job is proving it holds up with a tool that's *designed* to need it, not one that only fails by accident.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two tools whose jobs sound similar is the single most common way this step's test ("an unclear prompt, could apply to either tool") goes wrong in practice — and the fix isn't in the loop's code at all, it's in the tool *descriptions*, the docstring or `description=` argument that becomes what the model actually reads when deciding. A calculator and a "solve this equation" tool, or two search tools over slightly different data, will get chosen inconsistently if their descriptions both just say roughly "does math" or roughly "searches things." Sharpen each one to say specifically what it's *for* and, just as importantly, what it's *not* for — "use this for arithmetic and unit conversions; do NOT use this for looking up facts" reads very differently to the model than "a calculator."

The other real gap at this scale: `MaxIterationsExceeded` needs to carry enough for someone to actually debug the run afterward, not just announce that it happened. A bare `raise` `MaxIterationsExceeded()` with no context forces whoever's debugging to re-run the exact same failing prompt and hope it fails the same way twice (it might not, since these calls aren't perfectly deterministic). Attach the full `steps` log to the exception itself, so it's captured wherever this error gets caught and logged — one clean stack trace with the whole story attached, not a mystery to reproduce.

The extra pieces: - Tool `description`s written as "use this when X; do not use this for Y," reviewed specifically for overlap with your other tools' descriptions. - `class` `MaxIterationsExceeded(Exception):` with an `__init__` that accepts and stores the partial `steps` list (and the original `task`), so `except MaxIterationsExceeded as e: log(e.steps)` is possible at the call site. - A round counter checked *before* calling the model each time, not just after — so the very last round that would exceed the limit never even makes the API call.

Sketch two tool descriptions that could plausibly overlap, then sharpen them, before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get the mechanics working — more tools, a counter, a custom exception, a failing tool. Advanced is about the two ways this step's own test cases are designed to expose a shallow implementation: an unclear prompt reveals whether your tool descriptions actually disambiguate anything, and a step-limit hit reveals whether your error carries enough to debug from, or just enough to know something went wrong.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define 2 more tools, at least one that calls a real service, one that fails on a

function run_agent(task, max_iterations):
    messages = [system, task]
    round_count = 0
    loop:
        round_count += 1
        if round_count > max_iterations:
            raise MaxIterationsExceeded with the log so far

    ask the model, with all 3 tools available
    if no tool calls: return the final answer
    for each requested tool call:
        try to run it, catch any error
        record what happened either way
        feed the result back as a tool message
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
tools.py:
    calculator                (from Step 2, unchanged)
    lookup_weather(city) -> reuses Doc02's HTTP client
    flaky_lookup(order_id) -> raises on purpose if order_id == "BAD-ORDER"

agent.py:
    class MaxIterationsExceeded(Exception):
        def __init__(self, task, steps):
            super().__init__(f"Hit max_iterations on task: {task}")
            self.steps = steps

    def run_agent(task: str, max_iterations: int = 6) -> AgentResult:
        model = ChatOpenAI(model="gpt-4o-mini").bind_tools([calculator, lookup_weather])
        messages = [SystemMessage(...), HumanMessage(task)]
        steps: list[AgentStep] = []
        round_count = 0

        while True:
            round_count += 1
            if round_count > max_iterations:
                raise MaxIterationsExceeded(task, steps)

            response = model.invoke(messages)
            messages.append(response)
            if not response.tool_calls:
                return AgentResult(answer=response.content, steps=steps)

            for call in response.tool_calls:
                tool_fn = {"calculator": calculator, "lookup_weather": lookup_weather,
                           "flaky_lookup": flaky_lookup}[call["name"]]
                try:
                    result = tool_fn.invoke(call["args"])
                    ok = True
                except Exception as e:
                    result = f"Error: {e}"
                    ok = False
                steps.append(AgentStep(tool=call["name"], args=call["args"], result=result, ok=ok))
                messages.append(ToolMessage(content=result, tool_call_id=call["id"])
```

Write the full typed version yourself, with real logging (Doc01's `logger`) inside the loop for every round, before moving to the Advanced version.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

each tool's docstring/description states what it's for AND what it's not for

MaxIterationsExceeded carries task + full steps log, not just a message

round limit checked before the model call, not only after

Here's almost the whole thing — fill in the missing piece yourself:

```
@tool(args_schema=WeatherArgs)
def lookup_weather(city: str) -> str:
    """Look up the current weather for a named city. Use this ONLY for weather –
    do NOT use this for general facts about a place, like population or capital."""
    ...

@tool(args_schema=OrderArgs)
def flaky_lookup(order_id: str) -> str:
    """Look up an order's status by ID. Use this ONLY for order status –
    do NOT use this for anything unrelated to a specific order."""
    if order_id == "BAD-ORDER":
        raise RuntimeError("order service timed out")
    # your turn: return a plausible fake status for any other order_id
    ...
```

Fill in `flaky_lookup`'s success case, write `MaxIterationsExceeded` with the full context it needs, and wire the round-count check to run before the model call — then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: the loop's shape is identical across all 3 — Basic and Intermediate get it running with real tools, a limit, and recovery. Advanced is entirely about making the two things this step is actually testing (tool-choice clarity, and a debuggable limit-hit) hold up under the README's own test cases, not just under a tidy happy-path run.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)